

1. Dice Throw Problem using Dynamic Programming

```
main.c
11
12     for (int d = 1; d <= num_dice; d++) {
13         for (int sum = 1; sum <= target; sum++) {
14             for (int face = 1; face <= num_sides; face++) {
15                 if (sum >= face)
16                     dp[d][sum] += dp[d - 1][sum - face];
17             }
18         }
19     }
20
21     return dp[num_dice][target];
22 }
23
24 int main() {
25     int sides, dice, target;
26
27     printf("Enter number of sides: ");
28     scanf("%d", &sides);
29     printf("Enter number of dice: ");
30     scanf("%d", &dice);
31     printf("Enter target sum: ");
32     scanf("%d", &target);
33
34     printf("Number of ways to reach sum %d: %d\n",
35           target, diceThrow(sides, dice, target));
36
37     return 0;
38 }
```

input

```
Enter number of sides: 6
Enter number of dice: 2
Enter target sum: 7
Number of ways to reach sum 7: 6

...Program finished with exit code 0
Press ENTER to exit console.
```

2. Assembly Line Scheduling using DP

```
main.c
22
23 int dp[LINES][STATIONS];
24
25 for (int i = 0; i < LINES; i++)
26     dp[i][0] = a[i][0];
27
28 for (int j = 1; j < STATIONS; j++) {
29     for (int i = 0; i < LINES; i++) {
30         dp[i][j] = 99999;
31
32         for (int k = 0; k < LINES; k++) {
33             int time = dp[k][j - 1] +
34                 transfer[k][i] + a[i][j];
35
36             dp[i][j] = min(dp[i][j], time);
37         }
38     }
39 }
40
41 int result = dp[0][STATIONS - 1];
42
43 for (int i = 1; i < LINES; i++)
44     result = min(result, dp[i][STATIONS - 1]);
45
46 printf("Minimum production time: %d\n", result);
47
48 return 0;
49 }
```

Minimum production time: 17

...Program finished with exit code 0
Press ENTER to exit console.

3. Three Assembly Lines – Minimum Production Time

```
main.c
22         tsp(graph, mask | (1 << city),
23             city, dp);
24         ans = min(ans, cost);
25     }
26 }
27
28     return dp[pos][mask] = ans;
29 }
30
31 int main() {
32     int graph[N][N];
33     int dp[N][1 << N];
34
35     printf("Enter the 4x4 distance matrix:\n");
36
37     for (int i = 0; i < N; i++)
38         for (int j = 0; j < N; j++)
39             scanf("%d", &graph[i][j]);
40
41     for (int i = 0; i < N; i++)
42         for (int j = 0; j < (1 << N); j++)
43             dp[i][j] = -1;
44
45     printf("Minimum path distance: %d\n",
46         tsp(graph, 1, 0, dp));
47
48     return 0;
49 }
```

input

```
10 0 35 25
15 35 0 30
20 25 30 0
Minimum path distance: 80

...Program finished with exit code 0
Press ENTER to exit console.
```

4. Minimum Path Distance using Matrix Form

```
main.c
31 scanf("%d", &t2[1]),
32
33 printf("Enter entry times e1 and e2: ");
34 scanf("%d %d", &e1, &e2);
35
36 printf("Enter exit times x1 and x2: ");
37 scanf("%d %d", &x1, &x2);
38
39 int dp1[n], dp2[n];
40
41 dp1[0] = e1 + a1[0];
42 dp2[0] = e2 + a2[0];
43
44 for (int i = 1; i < n; i++) {
45     dp1[i] = min(dp1[i - 1] + a1[i],
46                 dp2[i - 1] + t2[i - 1] + a1[i]);
47
48     dp2[i] = min(dp2[i - 1] + a2[i],
49                 dp1[i - 1] + t1[i - 1] + a2[i]);
50 }
51
52 int result = min(dp1[n - 1] + x1,
53                 dp2[n - 1] + x2);
54
55 printf("Minimum processing time: %d\n", result);
56
57 return 0;
58 }
```

input

```
1 2
Enter entry times e1 and e2: 10 12
Enter exit times x1 and x2: 18 7
Minimum processing time: 33

..Program finished with exit code 0
Press ENTER to exit console.
```

5. TSP with 5 Cities

```
main.c
43     {20, 0, 35, 25, 30},
44     {15, 35, 0, 30, 20},
45     {20, 25, 30, 0, 15},
46     {25, 30, 20, 15, 0}
47 };
48
49 for (int i = 0; i < N; i++)
50     for (int j = 0; j < N; j++)
51         graph[i][j] = input[i][j];
52
53 visited[0] = 1;
54 currentPath[0] = 0;
55
56 tsp(0, 1, 0);
57
58 printf("Shortest route: ");
59
60 for (int i = 0; i <= N; i++) {
61     printf("%c", 'A' + bestPath[i]);
62
63     if (i < N)
64         printf(" -> ");
65 }
66
67 printf("\nTotal distance: %d\n", bestCost);
68
69 return 0;
70 }
```

input

```
Shortest route: A -> B -> D -> E -> C -> A
Total distance: 85
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

6. Longest Palindromic Substring

```
main.c
17     dp[1][1] = 1;
18
19     for (int len = 2; len <= n; len++) {
20         for (int i = 0; i <= n - len; i++) {
21             int j = i + len - 1;
22
23             if (s[i] == s[j] &&
24                 (len == 2 || dp[i + 1][j - 1])) {
25
26                 dp[i][j] = 1;
27
28                 if (len > maxlen) {
29                     maxlen = len;
30                     start = i;
31                 }
32             }
33         }
34     }
35
36     printf("Longest palindromic substring: ");
37
38     for (int i = start; i < start + maxlen; i++)
39         printf("%c", s[i]);
40
41     printf("\n");
42
43     return 0;
44 }
```

input

Enter string: Longest palindromic substring: babab

...Program finished with exit code 0
Press ENTER to exit console.

7. Longest Substring Without Repeating Characters

main.c

```
11
12     int last[256];
13
14     for (int i = 0; i < 256; i++)
15         last[i] = -1;
16
17     int left = 0;
18     int maxLen = 0;
19
20     for (int right = 0; s[right] != '\0'; right++) {
21         unsigned char c = s[right];
22
23         if (last[c] >= left)
24             left = last[c] + 1;
25
26         last[c] = right;
27
28         int len = right - left + 1;
29
30         if (len > maxLen)
31             maxLen = len;
32     }
33
34     printf("Length of longest substring: %d\n", maxLen);
35
36     return 0;
37
38 }
```

input

Enter string: abcabcbcb
length of longest substring: 3

...Program finished with exit code 0
Press ENTER to exit console.

8. Word Break Problem

```
main.c
31     for (int j = 0; j < i; j++) {
32
33         if (dp[j]) {
34             char word[100];
35
36             strncpy(word, s + j, i - j);
37             word[i - j] = '\0';
38
39             for (int k = 0; k < n; k++) {
40                 if (strcmp(word, dict[k]) == 0) {
41                     dp[i] = true;
42                     break;
43                 }
44             }
45         }
46
47         if (dp[i])
48             break;
49     }
50 }
51
52 if (dp[len])
53     printf("Output: true\n");
54 else
55     printf("Output: false\n");
56
57 return 0;
58 }
```

input

Enter dictionary words:

leet

code

Output: true

...Program finished with exit code 0

Press ENTER to exit console.

9. Word Break using Given Dictionary

```
main.c
28     for (int j = 0; j < i; j++) {
29
30         if (dp[j]) {
31             char temp[100];
32
33             strncpy(temp, s + j, i - j);
34             temp[i - j] = '\0';
35
36             for (int k = 0; k < n; k++) {
37                 if (strcmp(temp, dict[k]) == 0) {
38                     dp[i] = true;
39                     break;
40                 }
41             }
42         }
43
44         if (dp[i])
45             break;
46     }
47 }
48
49 if (dp[len])
50     printf("Output: Yes\n");
51 else
52     printf("Output: No\n");
53
54 return 0;
55 }
```

input

Enter string: ilike
Output: Yes

...Program finished with exit code 0
Press ENTER to exit console.

10. Text Justification

```
main.c
56     },
57     },
58     printf("\n");
59 }
60 }
61 }
62
63 int main() {
64     char words[100][30];
65     int n, maxWidth;
66
67     printf("Enter number of words: ");
68     scanf("%d", &n);
69
70     printf("Enter words:\n");
71
72     for (int i = 0; i < n; i++)
73         scanf("%s", words[i]);
74
75     printf("Enter maximum width: ");
76     scanf("%d", &maxWidth);
77
78     printf("Justified text:\n");
79
80     justify(words, n, maxWidth);
81
82     return 0;
83 }
```

input

Justified text:
This is an
example for text
justification

...Program finished with exit code 0
Press ENTER to exit console.

11. WordFilter – Prefix and Suffix Search

```
main.c
35 }
36
37 int main() {
38     char words[MAX_WORDS][MAX_LEN];
39     int n;
40
41     printf("Enter number of words: ");
42     scanf("%d", &n);
43
44     printf("Enter words:\n");
45
46     for (int i = 0; i < n; i++)
47         scanf("%s", words[i]);
48
49     char pref[MAX_LEN], suff[MAX_LEN];
50
51     printf("Enter prefix: ");
52     scanf("%s", pref);
53
54     printf("Enter suffix: ");
55     scanf("%s", suff);
56
57     int result = wordFilter(words, n, pref, suff);
58
59     printf("Output: %d\n", result);
60
61     return 0;
62 }
```

input

```
apple
Enter prefix: a
Enter suffix: e
Output: 0

...Program finished with exit code 0
Press ENTER to exit console.
```

12. Floyd's Algorithm – All-Pairs Shortest Path

main.c

```
16 }
17
18 int main() {
19     int dist[N][N] = {
20         {0, 3, INF, INF},
21         {INF, 0, 1, 4},
22         {INF, INF, 0, 1},
23         {INF, INF, INF, 0}
24     };
25
26     printf("Distance matrix before Floyd's Algorithm:\n");
27     printMatrix(dist);
28
29     for (int k = 0; k < N; k++)
30         for (int i = 0; i < N; i++)
31             for (int j = 0; j < N; j++)
32                 if (dist[i][k] != INF && dist[k][j] != INF &&
33                     dist[i][j] > dist[i][k] + dist[k][j])
34                     dist[i][j] = dist[i][k] + dist[k][j];
35
36     printf("\nDistance matrix after Floyd's Algorithm:\n");
37     printMatrix(dist);
38
39     printf("\nShortest distance from City 1 to City 3 = %d\n",
40         dist[0][2]);
41
42     return 0;
43 }
```

input

```
INF INF 0 1
INF INF INF 0
```

Shortest distance from City 1 to City 3 = 4

```
...Program finished with exit code 0
Press ENTER to exit console.
```

13. Floyd's Algorithm – Router Link Failure

```
main.c
38     int u = -1;
39
40     for (int i = 0; i < n; i++) {
41         if (!visited[i] &&
42             (u == -1 || best[i] > best[u]))
43             u = i;
44     }
45
46     if (u == -1 || best[u] == 0)
47         break;
48
49     visited[u] = 1;
50
51     for (int v = 0; v < n; v++) {
52         if (graph[u][v] && !visited[v]) {
53             double newProb =
54                 best[u] * prob[u][v];
55
56             if (newProb > best[v])
57                 best[v] = newProb;
58         }
59     }
60 }
61
62 printf("Maximum probability: %.5f\n", best[end]);
63
64 return 0;
65 }
```

input

Enter number of nodes: 0.25000
Enter number of edges: Enter start and end: Maximum probability: 1.00000

...Program finished with exit code 0
Press ENTER to exit console.

14. Floyd's Algorithm – City Network

```
main.c
8      {0, 2, INF, INF, 0},
9      {INF, 0, 3, INF, 2},
10     {INF, INF, 0, 1, INF},
11     {INF, INF, INF, 0, 1},
12     {INF, INF, INF, INF, 0}
13 };
14
15 for (int k = 0; k < N; k++)
16     for (int i = 0; i < N; i++)
17         for (int j = 0; j < N; j++)
18             if (d[i][k] != INF && d[k][j] != INF &&
19                 d[i][j] > d[i][k] + d[k][j])
20                 d[i][j] = d[i][k] + d[k][j];
21
22 printf("Shortest distance matrix:\n");
23
24 for (int i = 0; i < N; i++) {
25     for (int j = 0; j < N; j++) {
26         if (d[i][j] == INF)
27             printf("INF ");
28         else
29             printf("%d ", d[i][j]);
30     }
31     printf("\n");
32 }
33
34 return 0;
35 }
```

input

```
INF 0 3 4 2
INF INF 0 1 2
INF INF INF 0 1
INF INF INF INF 0

...Program finished with exit code 0
Press ENTER to exit console.
```

15. Optimal Binary Search Tree

```
main.c
38         float current = left + right + sum;
39
40         if (current < cost[i][j]) {
41             cost[i][j] = current;
42             root[i][j] = r;
43         }
44     }
45 }
46
47
48 printf("\nMinimum OBST Cost = %.2f\n", cost[1][n]);
49
50 printf("\nCost Table:\n");
51 for (int i = 1; i <= n; i++) {
52     for (int j = i; j <= n; j++) {
53         printf("%.2f ", cost[i][j]);
54     }
55     printf("\n");
56 }
57
58 printf("\nRoot Table:\n");
59 for (int i = 1; i <= n; i++) {
60     for (int j = i; j <= n; j++) {
61         printf("%d ", root[i][j]);
62     }
63     printf("\n");
64 }
65
66 return 0;
67 }
```

input

```
Root Table:
1 2 3 3
2 3 3
3 3
4
...Program finished with exit code 0
```

16. OBST – Keys 10, 12, 16, 21

```
main.c
35 }
36
37 int main() {
38     char words[MAX_WORDS][MAX_LEN];
39     int n;
40
41     printf("Enter number of words: ");
42     scanf("%d", &n);
43
44     printf("Enter words:\n");
45
46     for (int i = 0; i < n; i++)
47         scanf("%s", words[i]);
48
49     char pref[MAX_LEN], suff[MAX_LEN];
50
51     printf("Enter prefix: ");
52     scanf("%s", pref);
53
54     printf("Enter suffix: ");
55     scanf("%s", suff);
56
57     int result = wordFilter(words, n, pref, suff);
58
59     printf("Output: %d\n", result);
60
61     return 0;
62 }
```

apple
Enter prefix: a
Enter suffix: e
Output: 0

...Program finished with exit code 0
Press ENTER to exit console.

17. Cat and Mouse Game

```
main.c
67     if (result == MOUSE) {
68         color[p][c][1] = MOUSE;
69         queue[rear++] =
70             (State){p, c, 1, MOUSE};
71     }
72 }
73 } else {
74     /* Previous turn was Cat */
75     for (int pc = 1; pc < n; pc++) {
76         if (!graph[pc][c])
77             continue;
78
79         if (color[m][pc][0] != DRAW)
80             continue;
81
82         if (result == CAT) {
83             color[m][pc][0] = CAT;
84             queue[rear++] =
85                 (State){m, pc, 0, CAT};
86         }
87     }
88 }
89 }
90
91 printf("Result: %d\n", color[1][2][0]);
92
93 return 0;
94 }
```

input

```
Enter number of nodes: 0
Enter adjacency matrix:
Result: 0
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

18. Maximum Probability Path

```
main.c
38 int u = -1;
39
40 for (int i = 0; i < n; i++) {
41     if (!visited[i] &&
42         (u == -1 || best[i] > best[u]))
43         u = i;
44 }
45
46 if (u == -1 || best[u] == 0)
47     break;
48
49 visited[u] = 1;
50
51 for (int v = 0; v < n; v++) {
52     if (graph[u][v] && !visited[v]) {
53         double newProb =
54             best[u] * prob[u][v];
55
56         if (newProb > best[v])
57             best[v] = newProb;
58     }
59 }
60 }
61
62 printf("Maximum probability: %.5f\n", best[end]);
63
64 return 0;
65 }
```

input

Enter number of nodes: 0.25000
Enter number of edges: Enter start and end: Maximum probability: 1.00000

...Program finished with exit code 0
Press ENTER to exit console.

19. Unique Paths in a Grid

```
main.c
4   int m, n;
5
6   printf("Enter number of rows: ");
7   scanf("%d", &m);
8
9   printf("Enter number of columns: ");
10  scanf("%d", &n);
11
12  long long dp[m][n];
13
14  for (int i = 0; i < m; i++)
15      dp[i][0] = 1;
16
17  for (int j = 0; j < n; j++)
18      dp[0][j] = 1;
19
20  for (int i = 1; i < m; i++) {
21      for (int j = 1; j < n; j++) {
22          dp[i][j] = dp[i - 1][j] +
23                    dp[i][j - 1];
24      }
25  }
26
27  printf("Number of unique paths: %lld\n",
28         dp[m - 1][n - 1]);
29
30  return 0;
31 }
```

input

```
Enter number of rows: 3
Enter number of columns: 2
Number of unique paths: 3

...Program finished with exit code 0
Press ENTER to exit console.
```

20. Number of Good Pairs

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n;
5
6     printf("Enter number of elements: ");
7     scanf("%d", &n);
8
9     int nums[n];
10
11     printf("Enter elements:\n");
12
13     for (int i = 0; i < n; i++)
14         scanf("%d", &nums[i]);
15
16     int count = 0;
17
18     for (int i = 0; i < n; i++) {
19         for (int j = i + 1; j < n; j++) {
20             if (nums[i] == nums[j])
21                 count++;
22         }
23     }
24
25     printf("Number of good pairs: %d\n", count);
26
27     return 0;
28 }
```

input

```
Enter number of elements: 6
Enter elements:
1 2 3 1 1 3
Number of good pairs: 4

...Program finished with exit code 0
Press ENTER to exit console.
```

21. Find City with Smallest Number of Neighbors

```
main.c
52     }
53 }
54 }
55
56 int answer = -1;
57 int minCount = INF;
58
59 for (int i = 0; i < n; i++) {
60     int count = 0;
61
62     for (int j = 0; j < n; j++) {
63         if (i != j && dist[i][j] <= threshold)
64             count++;
65     }
66
67     printf("City %d has %d neighboring cities\n",
68           i, count);
69
70     if (count <= minCount) {
71         minCount = count;
72         answer = i;
73     }
74 }
75
76 printf("Output: %d\n", answer);
77
78 return 0;
79 }
```

input

```
City 1 has 3 neighboring cities
City 2 has 3 neighboring cities
City 3 has 2 neighboring cities
Output: 3

...Program finished with exit code 0
Press ENTER to exit console.
```